

PolicyModel (run133) — Decision Model + Trading Constraints Deep Dive

This write-up documents **run133** as a *learned portfolio allocator* that consumes cached alpha signals and produces **constrained long-only weights** under explicit trading and risk controls. The canonical implementation lives in `scripts/run133_train_portfolio.py`.

0) Executive summary

What it is: A **cross-sectional transformer policy** that maps per-ticker features (alpha + simple regime context + position hysteresis) and global market context into:

1. **per-ticker scores** $\rightarrow$ transformed into **long-only portfolio weights** via a constrained softmax pipeline; and
2. a learned **risk_gate** $\in [0,1]$ that scales the whole portfolio exposure.

What it optimizes: A differentiable episode objective targeting **excess Sharpe vs SPY** (when provided), with embedded turnover costs and additional penalties/bonuses for **breadth control (EffK)**, **alpha alignment**, and **drawdown control**.

1) Inputs and feature set

1.1 Per-ticker (cross-sectional) features

Computed by `precompute_static_features()` and augmented at runtime.

Static (cached per day, per ticker)

Per ticker, per day:

- **mu**: predicted expected return (primary signal from the alpha model).
- **mu_rank**: cross-sectional rank of mu in $[0, 1]$ (scale-invariant stabilization).
- **ret_1d, ret_5d, ret_20d**: rolling realized returns (regime/context for momentum vs mean-reversion).
- **live**: boolean mask for valid tickers on that date (prevents padded/inactive names from influencing decisions).

Static tensor shapes:

- `x_static`: $[T, N, 6] = (\text{mu}, \text{mu_rank}, \text{ret_1d}, \text{ret_5d}, \text{ret_20d}, \text{live})$

Runtime (added during unroll)

- **held_prev**: binary flag $1\{w_{\text{prev}} > 0\}$ indicating whether the name was held yesterday.

Runtime decision feature matrix per day:

- $x_t: [N, 7] = X_{\text{static}}[t, :, 0:5] + \text{held_prev} + \text{live}$

Why held_prev exists: it provides *hysteresis* so the model can learn to avoid unnecessary churn (especially because turnover cost is in the PnL).

1.2 Global (per-day) features

Also computed in `precompute_static_features()`:

- **mu_mean, mu_std, mu_max, mu_min**: cross-sectional distribution stats of mu over live names (signal dispersion/risk regime proxy).
- **ret_mean**: cross-sectional mean of 1d realized return (market regime proxy).

Shape:

- $g: [T, 5]$
-

2) Policy architecture (CrossSectionPolicy)

The policy is a transformer-based cross-sectional scorer with a separate risk head.

2.1 Network blocks

Inputs:

- $x: [B, N, F=7]$
- $g: [B, G=5]$
- **live_mask**: $[B, N]$

Core:

1. **Embedding layer**: $\text{Linear}(F \rightarrow \text{hidden})$
2. **Transformer encoder**: cross-sectional attention across tickers, with padding via `key_padding_mask = ~live_mask`
3. **Global projector**: MLP mapping $g \rightarrow g_{\text{emb}}$ (dimension = hidden)

Outputs:

- **Score head:** per-ticker score from concatenated $[h_i, g_emb]$, output shape $[B, N]$
- **Risk head:** scalar gate from g_emb , output shape $[B]$, via sigmoid:
 - $risk_gate = \text{sigmoid}(risk_head(g_emb)) \in (0, 1)$

2.2 Training-time score noise (optional)

If $noise_std > 0$, Gaussian noise is added to scores during training **only on live names**. This encourages exploration / robustness by preventing brittle score ordering.

3) Score $\rightarrow$ weight transformation (constraints pipeline)

Implemented in `scores_to_weights()` and applied inside the episode unroll.

Given per-ticker scores and `live_mask`, weights are produced as follows:

1. Temperature scaling

$$\text{logits} = \frac{\text{scores}}{\max(\text{temperature}, 1e-6)}$$

Lower temperature $\Rightarrow$ more concentrated allocations.

2. **Live mask** Non-live names get $-1e9$ logits (effectively excluded).
3. **Top-k cap (optional hard cap on active names)** If `k_cap` is set, only the top-k logits are kept; the rest are masked out pre-softmax.
4. **Softmax (long-only weights)**

$$w = \text{softmax}(\text{logits})$$

This ensures **long-only** and **sum-to-one** over active names.

5. Gross exposure scaling

$$w \leftarrow w \cdot \text{gross}$$

So that $\text{sum}(w) \approx \text{gross}$ (before risk gating).

6. **Per-name max weight (optional)** If `w_max` is set, clip each weight to `w_max` and renormalize back to `gross`.

7. Risk gate application (run133-specific)

$$w_{\text{final}} = w \cdot \text{risk_gate}$$

This is a scalar exposure shrink/expand mechanism in $[0,1]$.

Net effect: constrained, long-only, top-k limited, optional max-position-limited portfolio with explicit overall exposure control via `gross` and `risk_gate`.

4) Episode objective (what is optimized)

Implemented in `episode_objective()` as a deterministic differentiable unroll over the episode.

4.1 Per-day PnL definition

For day `t`, the policy produces `w_t` from features at `t`, then PnL uses next day returns:

$$\text{pnl} * t = \sum_i w * t, i \cdot \text{ret} * t + 1, i; -; \text{bench} * t + 1; -; \text{cost} \cdot \text{turnover}_t$$

Where:

- `bench_{t+1}` is `spy_ret[t+1]` if available, else `0`.
- Turnover is:

$$\text{turnover} * t = 0.5 \sum_i |w * t, i - w_{t-1, i}|$$

- Transaction cost uses `cost = cost_bps × 1e-4` multiplied by turnover.

4.2 Sharpe (primary objective)

The episode computes:

- `mean = E[pnl]`
- `vol = Std[pnl]`
- annualized:

$$\text{sharpe} = \frac{\text{mean}}{\text{vol}} \sqrt{252}$$

The base loss starts as:

$$\text{loss} = -\text{sharpe}$$

4.3 Breadth control: Effective K (EffK) penalty

EffK is computed as:

$$\text{eff_k} = \frac{(\sum_i |w_i|)^2}{\sum_i w_i^2 + \epsilon}$$

This is a smooth measure of diversification/breadth.

If **target_k** is set:

$$\text{coverage_penalty} = \lambda_{\text{effk}} \cdot \max(0, \text{eff_k_mean} - \text{target_k})$$

Interpretation: penalize portfolios that are **too broad** beyond a soft target.

4.4 Alpha alignment bonus

Defined as:

$$\alpha_{\text{mean}} = E_t \left[\sum_i w_{t,i} \cdot \mu_{t,i} \right]$$

In the loss:

$$-\lambda_{\alpha} \cdot \alpha_{\text{mean}}$$

Meaning higher alpha loading reduces the loss (encourages following the alpha model).

4.5 Drawdown penalty (optional)

Wealth process:

$$W_t = \prod_{s \leq t} (1 + \text{pnl}_s)$$

Max drawdown magnitude is computed and penalized if above **dd_target**:

$$\text{dd_excess} = \max(0, \text{max_dd} - \text{dd_target})$$

$$+\lambda_{\text{dd}} \cdot \text{dd_excess}$$

4.6 Full loss

$$\text{loss} = -\text{sharpe} * \lambda_{\text{effk}} \max(0, \text{eff_k_mean} - \text{target_k}) - \lambda_{\alpha} \alpha_{\text{mean}} * \lambda_{\text{dd}} \max(0, \text{max_dd} - \text{dd_target})$$

Turnover cost is included inside pnl, not as a separate penalty term.

5) Risk gate semantics (run133 differentiator)

`risk_gate` is a learned scalar that depends only on global features `g_t`. It serves as:

- a **regime-based exposure controller** (e.g., dial down exposure when dispersion is poor or market regime is adverse),
- a differentiable alternative to hard risk constraints, and
- a mechanism to trade off Sharpe vs drawdowns implicitly.

Operationally, it scales weights after the constrained softmax transformation:

- constraints determine **composition**,
- risk gate determines **overall size**.

6) Correct evaluation logic (important implementation note)

Your current evaluator (`scripts/eval_run130_policy.py`) is **not run133-correct**, because:

1. It loads a policy definition that **does not include the risk head**, and
2. It does not apply `risk_gate` to weights.

6.1 What run133 evaluation must do

The run133 evaluator should:

- Load `CrossSectionPolicy` from `scripts/run133_train_portfolio.py`.
- Forward pass: `scores, risk_gate = policy(x, g, live)`
- Convert scores to weights: `w_raw = scores_to_weights(scores, ...)`
- Apply gate: `w = w_raw * risk_gate`

This is consistent with the logic already used inside `episode_objective()` in the trainer.

6.2 Minimal remediation path

Create `scripts/eval_run133_policy.py` by copying `eval_run130_policy.py` and:

- Import from `run133_train_portfolio` (not `run129_train_portfolio`)
 - Update the forward path to unpack (`scores`, `risk_gate`)
 - Multiply the constrained weights by `risk_gate`
 - Log `risk_gate` daily
-

7) Recommended evaluation output spec (auditable + plot-friendly)

7.1 `daily.csv` (per day)

Include at minimum:

- `date`
- `portfolio_return` (raw)
- `spy_return`
- `excess_return` (`portfolio_return` - `spy_return`)
- `turnover`
- `eff_k`
- `risk_gate`

7.2 `trades.csv` (optional per ticker)

- `date`
- `ticker`
- `w_prev`, `w_new`, `delta_w`
- `return`
- `pnl_gross`, `cost_alloc`, `pnl_net`

7.3 `summary.json` (optional)

- Sharpe (raw and excess)
 - max drawdown
 - mean turnover
 - mean effK
 - beta to SPY (optional; requires explicit computation)
-

8) Known gaps / open items (per provided notes)

- There is **no dedicated run133 eval script** applying `risk_gate` in the repo (needs to be added).
 - Existing plots based on `eval_run130_policy.py` will **misstate** run133 performance because they ignore `risk_gate`.
 - Beta-to-SPY is **not computed** in the existing scripts and would need to be added if desired for reporting.
-

9) Operational parameterization (what materially changes behavior)

Key knobs and their effects:

- `temperature`: lower $\Rightarrow$ sharper allocations, more concentration, higher turnover sensitivity.
 - `k_cap`: hard cap on names; lower $\Rightarrow$ more concentrated and easier-to-audit portfolios.
 - `w_max`: prevents single-name dominance; interacts with `temperature` and `k_cap`.
 - `gross`: target gross exposure prior to risk gate.
 - `cost_bps`: higher $\Rightarrow$ stronger implicit pressure to reduce turnover / stabilize holdings.
 - `lambda_effk` + `target_k`: controls breadth; higher penalty / lower target pushes concentration.
 - `lambda_alpha`: increases adherence to alpha; too high may increase churn if alpha is noisy.
 - `lambda_dd` + `dd_target`: adds explicit risk aversion to deep drawdowns.
 - `score_noise_std`: improves robustness; too high can degrade convergence.
-

If you want, I can also provide a “drop-in” `eval_run133_policy.py` skeleton consistent with your current `eval_run130_policy.py` logging format (including the `daily.csv` and `trades.csv` columns above), but the write-up above reflects the model as implemented in `run133_train_portfolio.py`.